Project 4: Multi-GPU Matrix-Vector Multiplication

Devansh Tripathi and Ashik S
SoM, IISER TVM

Instructor: Dr. Nagaiah Chamakuri
SoM, IISER TVM

April 16, 2026

Presentation Overview

1. CPU Baseline Implementation
2. GPU Compute Kernel
3. Multi-GPU Orchestration
4. Main Function & Memory Allocation
5. Performance Analysis

Serial Matrix-Vector Multiplication (CPU)

- Standard $O(N^2)$ algorithmic complexity.
- Computes $y = Ax$ sequentially row by row.
- Serves as our baseline for correctness verification and speedup calculations.

Serial Matrix-Vector Multiplication (CPU)

- Standard $O(N^2)$ algorithmic complexity.
- Computes $y = Ax$ sequentially row by row.
- Serves as our baseline for correctness verification and speedup calculations.

```
/* ---------------------------------------------------------
 * Serial Matrix-Vector Multiplication (CPU)
 * --------------------------------------------------------- */
void serialMatVec(const float* A, const float* x, float* y, int M,
    int N) {
    for (int i = 0; i < M; ++i) {
        float sum = 0.0f;
        for (int j = 0; j < N; ++j) {
            sum += A[i * N + j] * x[j];
        }
        y[i] = sum;
    }
}
```

Compute Kernel: Local Matrix-Vector Multiplication

- 1D grid layout maps each thread to a specific row of the matrix.
- Boundary check ensures threads outside the matrix dimension safely exit.

Compute Kernel: Local Matrix-Vector Multiplication

- 1D grid layout maps each thread to a specific row of the matrix.
- Boundary check ensures threads outside the matrix dimension safely exit.

```
/* ---------------------------------------------------------
 * Compute Kernel: Local Matrix-Vector Multiplication
 * --------------------------------------------------------- */
__global__ void matVecMulKernel(const float* A, const float* x,
    float* y, int numRows, int N) {
    // Calculate global row index for this thread
    int row = blockIdx.x * blockDim.x + threadIdx.x;

    // Boundary check to prevent memory access violations
    if (row < numRows) {
        float sum = 0.0f;
        for (int j = 0; j < N; ++j) {
            sum += A[row * N + j] * x[j];
        }
        y[row] = sum;
    }
}
```

Multi-GPU Orchestration: Setup & Initialization

- Detect available GPUs and calculate `rowsPerGPU` (Domain Decomposition).
- Allocate arrays to hold streams and device pointers.

Multi-GPU Orchestration: Setup & Initialization

- Detect available GPUs and calculate `rowsPerGPU` (Domain Decomposition).
- Allocate arrays to hold streams and device pointers.

```
float multiGpuMatVec(const float* h_A, const float* h_x, float* h_y
    , int M, int N) {
    int numGPUs;
    cudaGetDeviceCount(&numGPUs);

    if (numGPUs == 0) {
        printf("Error: No CUDA-capable devices found.\n");
        return 0.0f;
    }

    // Assuming M is perfectly divisible by the number of GPUs
    numGPUs = 1; // Explicit override for testing
    int rowsPerGPU = M / numGPUs;

    float** d_A = (float**)malloc(numGPUs * sizeof(float*));
    float** d_x = (float**)malloc(numGPUs * sizeof(float*));
    float** d_y = (float**)malloc(numGPUs * sizeof(float*));
    cudaStream_t* streams = (cudaStream_t*)malloc(numGPUs * sizeof(
        cudaStream_t));
```

Partition Matrix and Broadcast Vector

- **Scatter:** Each GPU gets a slice of Matrix A using pointer offsets.
- **Broadcast:** Every GPU requires the *entire* vector x.

Partition Matrix and Broadcast Vector

- **Scatter:** Each GPU gets a slice of Matrix A using pointer offsets.
- **Broadcast:** Every GPU requires the *entire* vector x.

```
/* Partition Matrix Rows and Broadcast Vector x */

for (int d = 0; d < numGPUs; ++d) {
    cudaSetDevice(d);
    cudaStreamCreate(&streams[d]);
    // Allocate memory on the current device
    cudaMalloc((void**)&d_A[d], rowsPerGPU * N * sizeof(float))
        ;
    cudaMalloc((void**)&d_x[d], N * sizeof(float));
    cudaMalloc((void**)&d_y[d], rowsPerGPU * sizeof(float));

    // Scatter row-blocks of A
    size_t offset_A = d * rowsPerGPU * N;
    cudaMemcpyAsync(d_A[d], h_A + offset_A, rowsPerGPU * N *
        sizeof(float), cudaMemcpyHostToDevice, streams[d]);

    // Broadcast the full vector x
    cudaMemcpyAsync(d_x[d], h_x, N * sizeof(float),
        cudaMemcpyHostToDevice, streams[d]);
}
```

Perform Local Computation

- Standard block/grid calculation based on local rows per GPU.
- Kernel is launched asynchronously within its respective device stream.

Perform Local Computation

- Standard block/grid calculation based on local rows per GPU.
- Kernel is launched asynchronously within its respective device stream.

```
/* ---------------------------------------------------------
 * Perform Local Computation
 * --------------------------------------------------------- */
int threadsPerBlock = 256;
int blocksPerGrid = (rowsPerGPU + threadsPerBlock - 1) /
    threadsPerBlock;

for (int d = 0; d < numGPUs; ++d) {
    cudaSetDevice(d);
    matVecMulKernel<<<blocksPerGrid, threadsPerBlock, 0,
        streams[d]>>>(
        d_A[d], d_x[d], d_y[d], rowsPerGPU, N
    );
}
```

Gather Results & Synchronization

- **Gather:** Pull computed y chunks back to the host, offsetting the destination pointer based on the GPU ID.
- **Sync:** CPU waits for all streams across all GPUs to complete.

Gather Results & Synchronization

- **Gather:** Pull computed y chunks back to the host, offsetting the destination pointer based on the GPU ID.
- **Sync:** CPU waits for all streams across all GPUs to complete.

```
/* ---------------------------------------------------------
 * Gather Results & Synchronization
 * --------------------------------------------------------- */
for (int d = 0; d < numGPUs; ++d) {
    cudaSetDevice(d);

    // Offset for the output vector y based on the GPU index
    size_t offset_y = d * rowsPerGPU;
    cudaMemcpyAsync(h_y + offset_y, d_y[d], rowsPerGPU * sizeof
        (float), cudaMemcpyDeviceToHost, streams[d]);
}

for (int d = 0; d < numGPUs; ++d) {
    cudaSetDevice(d);
    // Wait for all asynchronous operations on this stream to
        finish
    cudaStreamSynchronize(streams[d]);
}
```

Main Setup: The Pinned Memory Advantage

- Using `cudaMallocHost` creates **pinned (page-locked) memory**.
- This is strictly required for `cudaMemcpyAsync` to actually function concurrently across multiple streams without blocking the CPU.

Main Setup: The Pinned Memory Advantage

- Using `cudaMallocHost` creates **pinned (page-locked) memory**.
- This is strictly required for `cudaMemcpyAsync` to actually function concurrently across multiple streams without blocking the CPU.

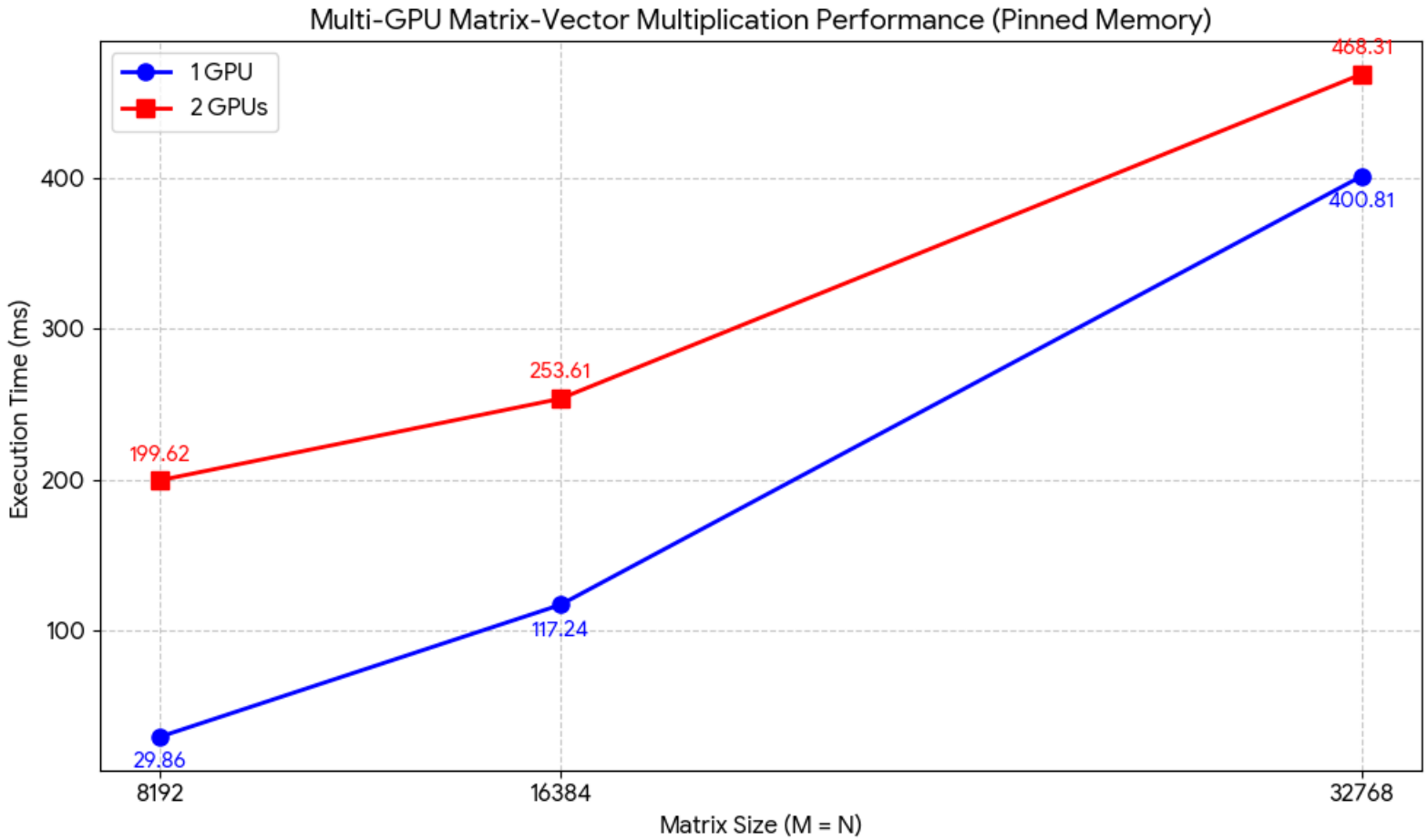
```
int main() {
    int M = 8192*2*2;
    int N = 8192*2*2;

    float *h_A, *h_x, *h_y_gpu;

    // Allocate pinned memory for asynchronous transfers
    cudaMallocHost((void**)&h_A, M * N * sizeof(float));
    cudaMallocHost((void**)&h_x, N * sizeof(float));
    cudaMallocHost((void**)&h_y_gpu, M * sizeof(float));
    float* h_y_cpu = (float*)malloc(M * sizeof(float));

    srand(42);
    for (int i = 0; i < M * N; ++i) h_A[i] = (float)rand() /
        RAND_MAX;
    for (int i = 0; i < N; ++i)     h_x[i] = (float)rand() /
        RAND_MAX;

    float gpu_ms = multiGpuMatVec(h_A, h_x, h_y_gpu, M, N);
```

Performance Comparison: 1 GPU vs. 2 GPUs

- Utilizing pinned memory (`cudaMallocHost`) successfully enables asynchronous execution.
- For operations with low arithmetic intensity like Mat-Vec, PCIe bandwidth becomes the primary bottleneck.

Performance Comparison: 1 GPU vs. 2 GPUs

- Utilizing pinned memory (`cudaMallocHost`) successfully enables asynchronous execution.
- For operations with low arithmetic intensity like Mat-Vec, PCIe bandwidth becomes the primary bottleneck.

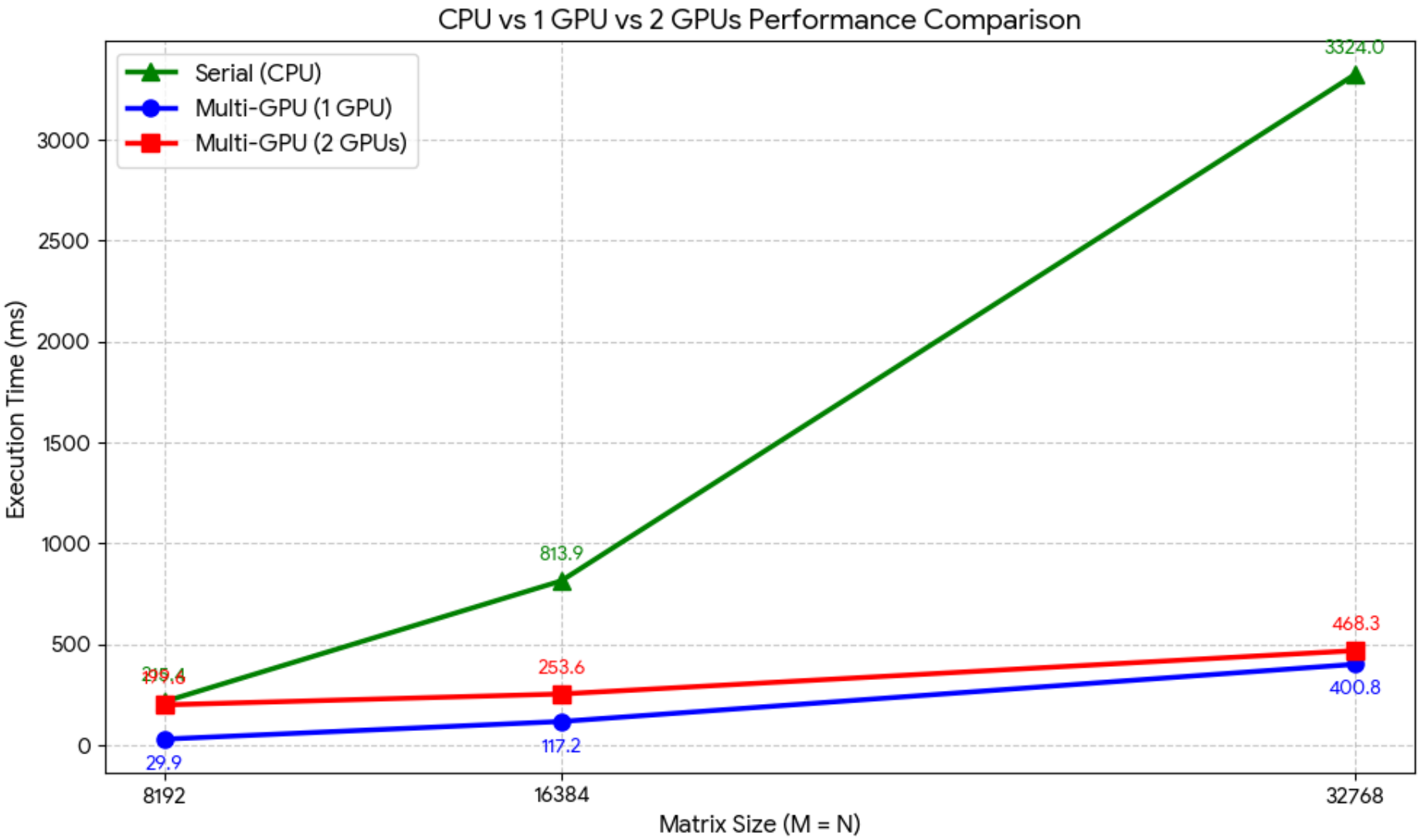


Scaling Results: CPU vs. Multi-GPU Speedup

- The CPU is faster for smaller matrices ($N = 8192$) due to the overhead of kernel launches and data transfers.

Scaling Results: CPU vs. Multi-GPU Speedup

- The CPU is faster for smaller matrices ($N = 8192$) due to the overhead of kernel launches and data transfers.



Performance Metrics: Speedup & Efficiency

Case Study: Largest Matrix ($N = 32768$), scaling from 1 GPU to 2 GPUs.

Performance Metrics: Speedup & Efficiency

Case Study: Largest Matrix ($N = 32768$), scaling from 1 GPU to 2 GPUs.

1. Multi-GPU Speedup (S):

- T_1 (Time with 1 GPU) $= 400.81$ ms
- T_2 (Time with 2 GPUs) $= 468.31$ ms

$$S = \frac{T_1}{T_2} = \frac{400.81}{468.31} \approx \mathbf{0.856}\times$$

Performance Metrics: Speedup & Efficiency

Case Study: Largest Matrix ($N = 32768$), scaling from 1 GPU to 2 GPUs.

1. **Multi-GPU Speedup (S):**
   - T_1 (Time with 1 GPU) $= 400.81$ ms
   - T_2 (Time with 2 GPUs) $= 468.31$ ms

$$S = \frac{T_1}{T_2} = \frac{400.81}{468.31} \approx \mathbf{0.856}\times$$

2. **Parallel Efficiency (E):**
   - P (Number of GPUs) $= 2$

$$E = \frac{S}{P} = \frac{0.856}{2} \approx \mathbf{0.428\ (or\ 42.8\%)}$$

References

NVIDIA Corporation.
CUDA Programming Guide.
[Online]. Available:
https://docs.nvidia.com/cuda/cuda-programming-guide/

Thank you!